

Tasks for Software Research & Development Engineer Role

Congratulations! We were impressed by your application and would like to invite you to complete one of the tasks below to help us better understand how you approach engineering, problem solving, and technical decision-making.

Please choose one task that you find interesting and spend approximately 6–8 hours working on it. When sharing your work, please include:

- what you implemented within the given timeframe,
- any assumptions or limitations,
- and any additional ideas, extensions, or improvements you would pursue with more time.

Please let us know when you expect to complete the task. Take your time and enjoy the process. If these kinds of problems feel engaging and motivating to you, there is a good chance you would thrive in our team and working environment.

Additional guidance on how we evaluate these tasks is provided below.

Table of contents:

Instructions to all tasks.....	2
What we care about.....	2
Task Descriptions.....	2
Task 1 - Clinical NLP Pipeline Design & Evaluation	2
Task 2 - Eclair Client Extension for Open-Source LLMs.....	4
Task 3 - Browser-Native ML App: Transcription, RAG & DuckDB.....	5
Task 4 - LOINC Code Validator Web App	6
Task 5 - SHACL-Driven Food Ordering API	8
Task 6 - GitHub Template for AI-Assisted Development Best Practices	10
Task 7 - Geospatial RAG + Knowledge Graph Dashboard.....	11

Instructions to all tasks

What we care about

We encourage you to choose a task that feels interesting and appropriately challenging rather than simply selecting the easiest option. We value curiosity, openness, and thoughtful engineering decisions, and believe learning often happens when stepping outside one's comfort zone.

To make these tasks engaging and realistic, we emphasize the following:

- **Quality over quantity** – better to have **less, well-written code than more, unreadable code**.
- **Learning over producing** – we encourage candidates to **learn more** about technologies and **share insights** rather than produce code without context.
- **Planning over executing** – we appreciate hearing or reading about your decisions and thought process behind an implementation.
- **Prioritization over completeness** – these tasks are intentionally scoped to encourage pragmatic decision-making. We appreciate clear reasoning about what you prioritized, what you intentionally left out, and how you would approach extensions or improvements with more time.
- **Transparency over results** – it's okay to struggle and seek help (human or AI); just share where and how you received assistance.

Task Descriptions

Task 1 - Clinical NLP Pipeline Design & Evaluation

Background

Biomedical NLP pipelines can be implemented using a variety of approaches, ranging from task-specific encoder models to modern instruction-tuned open-weight LLMs. Different solutions offer different trade-offs in terms of accuracy, interpretability, deployment complexity, privacy, reproducibility, and operational robustness in healthcare environments.

Examples of viable approaches include [OpenMed](#) library, or other biomedical models available on HuggingFace (<https://huggingface.co/models?sort=trending&search=biomedical+ner>). The objective of this task is not to use a specific framework, but to demonstrate sound technical judgment in selecting, testing, and evaluating an approach for clinical text processing.

Open-source biomedical natural language processing (NLP) has matured significantly. Libraries like [OpenMed](#) provide production-ready encoder models for named entity recognition across clinical domains, as well as multilingual personally identifiable information (PII) detection and de-identification. Unlike generative LLMs, these are extraction models: they identify and classify spans of text into structured output. This makes them well-suited for privacy-sensitive clinical pipelines where data must not leave a controlled environment.

The goal of this task is to propose and prototype an approach for clinical text processing, and to demonstrate initial testing and technical reasoning around the selected solution. We are more interested in sound architectural decisions and critical evaluation than in building an end-to-end pipeline.

Task

Propose and prototype an approach for processing clinical text through two stages:

- **De-identification.** Detect and redact personally identifiable information (PII) from a clinical note. You may use any reasonable open-source approach. The output of this stage should be a redacted version of the text suitable for downstream processing.
- **Entity extraction.** Apply one or more clinical entity extraction approaches to the de-identified text (e.g. disease detection, medication extraction, symptom detection, laboratory values). Produce structured output listing detected entities, their types, text spans, and confidence scores.

The prototype should be runnable from the command line or via a minimal API. Use synthetic or publicly available clinical text as input.

Include a short written section (max. one page) describing your reasoning behind the selected approach, alternative approaches considered, where human review would be required in the pipeline and why, and the potential failure modes of automated de-identification in a production healthcare setting.

Format

Present your work in a repository (GitHub or GitLab). The project must be reproducible, well-documented, and include a clear README with setup instructions and example input/output.

A production-ready end-to-end system is not expected. We are more interested in sound technical reasoning, architectural choices, and the ability to prototype and evaluate an approach under realistic time constraints.

What We Will Be Looking At

- **Technical and architectural choices:** The ability to pragmatically prototype and evaluate a selected approach, with clear awareness of the trade-offs between different approaches.
- **Pipeline design:** A clean, modular, and reproducible pipeline design with appropriately separated processing stages.
- **Approach to testing:** Initial evaluation of the selected approach, including awareness of limitations, robustness, uncertainty, and handling of edge cases.
- **Reflection and critical thinking:** Clear discussion of assumptions, risks, and the limitations of automated NLP in clinical settings.
- **Reproducibility:** Ease of running the prototype on new input with minimal or no modification.

Task 2 - Eclair Client Extension for Open-Source LLMs

Background

[Croissant](#) is a JSON-LD metadata format for machine learning datasets, maintained by MLCommons and adopted by HuggingFace, Kaggle, OpenML, and Google Dataset Search. It describes datasets in a standardized, machine-readable way, covering resources, data structure, splits, and ML semantics.

[Eclair](#) is an agentic layer on top of Croissant. It provides an MCP server that exposes tools for LLMs to search, retrieve, and reason over dataset metadata. The Python implementation already ships client adapters for Gemini and Claude, which handle the tool-call loop, message formatting, and MCP session lifecycle for those providers.

The goal of this task is to extend the Eclair client interface to support open-source LLMs served locally or via open inference endpoints.

Task

Study the existing client implementations for Gemini and Claude in the Eclair repository. Identify the interface they implement, i.e. the message contract, tool-call loop structure, and provider assumptions.

Then implement a new client that targets at least one open-source LLM of your choice, served via a locally running inference backend (e.g. Ollama, vLLM, llama.cpp with OpenAI-compatible mode). The model must support tool/function calling. Your client must implement the same interface as the existing ones, and must work end-to-end: a user query that triggers at least one Croissant MCP tool call and returns a grounded answer.

Include a short written section covering: what assumptions the existing client interface makes about the provider, which your chosen model satisfies natively, and where you had to adapt or work around differences.

If time allows: extract a shared base class or interface that unifies your new client with the existing ones, reducing duplication.

This task is designed for approximately 6–8 hours. Prioritize a minimal working prototype, clear reasoning behind the implemented features, and a concise discussion of possible extensions or improvements that could be added with more time.

Format

Present your work as a pull-request-style addition to a fork of the Eclair repository. Include a clear README section on how to set up the local inference backend and run the new client. The work must be reproducible without access to any paid API.

What We Will Be Looking At

- **Codebase comprehension:** does the candidate read and understand the existing interface before writing new code?
- **Correctness of the tool-call loop:** do tool results round-trip properly back to the model?
- **Provider adaptability:** how does the candidate handle differences between the open-source model's API and what the existing interface assumes?
- **Written analysis:** does the candidate surface the right abstractions and tradeoffs?
- **Reproducibility:** can we run the full flow locally without a proprietary API key?

Task 3 - Browser-Native ML App: Transcription, RAG & DuckDB

Background

Browser-native machine learning has become practical. Libraries like [transformers.js](#) and [DuckDB-WASM](#) make it possible to run inference and query databases entirely in the browser, without a server. At the same time, some tasks using larger models, persistent storage, or complex queries still belong server-side. Knowing where to draw that line is a meaningful engineering decision.

The goal of this task is to build a static web application that combines three ML-powered capabilities, and to articulate a principled architecture for deciding when to use local versus remote model deployment.

Task

Build a static web application, meaning without backend you own or server-side rendering, with the following three features:

- **Audio transcription.** The user records or uploads a short audio clip and receives a transcript. The audio must not leave the browser.
- **Agentic RAG chat.** The user can ask questions over a small document corpus of your choice. Retrieval happens locally; the LLM call for the final answer may be remote. Retrieved source chunks should be visible in the UI alongside the answer.
- **Database interface.** The user can run analytical queries against a dataset of your choice using DuckDB-WASM. At least one query should aggregate or join data. Results are displayed in the UI.

The choice of models, datasets, document corpus, and UI framework is yours.

Include a written architecture document that defines a decision framework for when each capability runs locally versus remotely. Apply that framework to your own implementation, and describe at least one scenario per feature where you would flip the decision.

A thoughtful GUI with clear loading states, graceful degradation, a coherent layout across the three features will be evaluated positively.

This task is designed to be completed in approximately 6–8 hours. Prioritize a clear and functional prototype, sound architectural decisions, and thoughtful reasoning about trade-offs. If there are features, extensions or improvements you do not have time to implement, briefly describe how you would approach them.

Format

Present your work in a repository (GitHub or GitLab). The app must be deployable as a static site (e.g. via npm run build). Include a clear README with setup instructions and a note on any remote endpoints required to run the full application.

What We Will Be Looking At

- **Technical and architectural choices:** Clear reasoning about local versus remote execution trade-offs and the ability to pragmatically prototype and evaluate these decisions.
- **Correctness:** Transcription runs fully in-browser, retrieval happens locally, and DuckDB queries execute client-side without a server.
- **User experience and robustness:** Loading states, error handling, graceful degradation, and responsiveness are handled thoughtfully.
- **Code structure:** The codebase is modular, understandable, and reasonably extensible.
- **Reflection and critical thinking:** Assumptions, trade-offs, limitations, and future improvements are clearly articulated.
- **Reproducibility:** The application can be built and run from a clean checkout with minimal effort.

Task 4 - LOINC Code Validator Web App

Background

Health institutions need to digitalize many types of health-related data, including medical procedures, laboratory tests, drugs, active principles, and vital signs. A common best practice is to represent such data using national or international standards. LOINC (<https://loinc.org>) is a widely used international terminology for laboratory tests, measurements, and clinical observations, including vital signs such as heart rate.

The National Institute of Health from the US offers an API that allows for users to check various aspects related to a specific LOINC code:

<https://clinicaltables.nlm.nih.gov/apidoc/loinc/v3/doc.html>

Modern languages and frameworks such as Rust, Go offer lightweight ways to build portable web applications. Examples of technologies commonly used in these ecosystems include WebAssembly (<https://webassembly.org/>) and HTMX (<https://htmx.org/>).

The goal of this task is to build a small web application that allows a user to check whether a LOINC code is valid, and to briefly reflect on the strengths and limitations of the chosen implementation approach.

We provide optional extension ideas if time permits. If you do not have time to implement them, feel free to briefly describe additional ideas, improvements, or future extensions you would consider.

We recommend using Rust or Go since they offer streamlined ways to handle both backend and frontend development together, but another stack is acceptable if the choice is justified.

Task

Build a web application where a user can enter a LOINC code and validate it by clicking a button.

The application should:

- accept a LOINC code as input;
- call the LOINC API (<https://clinicaltables.nlm.nih.gov/apidoc/loinc/v3/doc.html>) to check whether the code exists;
- display whether the code is valid or invalid;
- handle basic edge cases such as empty input, malformed codes, API errors, or no results;
- provide clear feedback to the user.

The app may be implemented in Go or Rust. Possible starting points include:

Rust Resources:

- <https://leptos.dev> (good documentation to get started in a very Rusty format)
- <https://dioxuslabs.com> (detailed documentation with front-end jargon, including many examples)
- <https://rocket.rs> + <https://htmx.org>

Go Resources:

- Go standard library provides building blocks such as `html/template` and `net/http`.
- <https://htmx.org/>

Additional ideas (optional):

If time allows, you may implement or describe one or more of the following:

- Small: upload a file with a list of LOINC codes and output a validation report;
- Small: display additional information about a valid code, such as name, LOINC version, or related terms;

- Small: export validation results with metadata, such as date and time of check and LOINC version;
- Medium: deploy the application on GitHub, GitLab, or another platform;
- Fancy: support multiple medical coding systems with other NIH APIs (<https://clinicaltables.nlm.nih.gov/>);
- Fancy: suggest similar codes when there is no exact match.

You are not expected to implement all of these. Clear prioritization and reasoning are valued more than feature completeness.

Format

Present your work in a repository (GitHub or GitLab). Follow development best practices: the project must be reproducible, well-documented, and include a clear README with local setup instructions together with strengths and limitations of your code/approach/framework.

What We Will Be Looking At

- **Functionality and usability:** The app validates LOINC codes correctly and gives clear feedback to the user.
- **Code structure:** Components are easy to identify, understand, and maintain.
- **API integration and error handling:** The app handles valid codes, invalid codes, malformed input, and API errors appropriately.
- **Technical choices:** The selected framework and implementation approach are justified.
- **Reproducibility:** The repository can be cloned and run without guesswork.
- **Reflection:** Strengths, limitations, and possible improvements are clearly described.

Task 5 - SHACL-Driven Food Ordering API

Background

The goal of this task is to build a small but complete food-ordering application where the backend drives the form logic using **JSON-LD** documents and **SHACL shapes**. The frontend is **optional** and should be minimal and simple. Its only job is to display what the backend provides. The backend can be written in a language of your choice.

Task

Build a prototype service that exposes a small API for configuring and ordering two dishes: **French Tacos** and **Ramen**.

The backend should store the JSON-LD documents and SHACL shapes for each dish. The SHACL shapes should define the relevant form constraints, such as required fields, allowed values, cardinality, and simple validation rules.

The backend should expose the form definition in a machine-readable format consumable by a frontend, for example JSON Forms schema + UI schema. The translation from SHACL/JSON-LD to this format should happen exclusively on the backend.

The backend should expose at minimum:

- an endpoint to retrieve the form schema for a given dish;
- an endpoint to submit and validate a filled-in form.

Validation should happen on the backend. When the user submits the form, the backend validates the payload against the SHACL shapes and returns structured errors if constraints are violated.

The frontend is **optional**. If implemented, it should be minimal: it should allow the user to select a dish, render the schema returned by the backend, submit the form data, and display either confirmation or validation errors. The frontend should not contain dish-specific form logic or definitions. No persistence is required.

If time allows, support a third dish of your choice, added purely through a new JSON-LD document and SHACL shape on the backend, with zero frontend changes required. This will demonstrate the strength of the architecture.

Include a short written section explaining your SHACL/JSON-LD modelling choices, how the translation to the frontend form schema works, and any limitations or assumptions.

Format

Present your work in a repository (GitHub or GitLab). Follow development best practices: the project must be reproducible, well-documented, and include a clear README with setup instructions.

What We Will Be Looking At

- **Backend architecture:** Clean, well-structured SHACL/JSON-LD processing that is easy to extend.
- **Separation of concerns:** Adding a new dish should not require frontend changes or hardcoded frontend form logic.
- **Validation:** SHACL-based validation should be meaningful, with clear structured errors.
- **Approach to testing:** Evidence that valid and invalid submissions were tested.
- **Reproducibility:** The repository can be cloned and run without guesswork.

Task 6 - GitHub Template for AI-Assisted Development Best Practices

Background

AI-assisted coding is increasingly used in software development, but its effectiveness depends strongly on project structure, toolchain isolation, clear development conventions, and automated quality checks. A good repository template should help developers use AI tools productively while reducing risks such as inconsistent code style, hallucinated dependencies, insecure patterns, weak tests, or unreproducible environments.

The goal of this task is to propose and prototype a repository template that supports AI-assisted coding using sound software engineering practices. The template may be generic or opinionated, but design choices should be clearly justified.

Task

Build a repository template that provides a basic setup for AI-assisted development.

You can, but don't have to, use our templates as a starting point or inspiration:

- <https://github.com/sdsc-ordes/repository-template>

Your template should include:

- toolchain isolation, for example via a devcontainer, sandbox, or similar;
- configuration or documentation to steer AI-assisted coding behavior, such as coding rules, project context, model-specific instructions, or contribution guidelines;
- automated checks to validate AI-generated output, such as formatting, linting, type checking, tests, security checks, dependency checks, or CI workflows;
- clear user guidance explaining how to use the template and how the AI-related safeguards are intended to work.

The template should be reasonably generic, but you may include language-, IDE-, or framework-specific sub-templates if useful. Tools such as Copier, Cookiecutter, or similar are welcome if they help make the template flexible.

Format

Present your work in a repository (GitHub or GitLab). The project must be reproducible, well-documented, and include a clear README with instructions. Add a short written section explaining your design choices, assumptions, and what you would improve with more time.

What We Will Be Looking At

- **Technical and architectural choices:** The ability to pragmatically prototype and justify a repository template for AI-assisted coding, with clear awareness of trade-offs between different tools and workflows.

- **AI-readiness and safeguards:** Clear mechanisms to steer AI behavior and validate AI-generated output through documentation, rules, tests, and automated checks.
- **Structure and usability:** A template that is easy to understand, instantiate, and adapt to realistic development scenarios.
- **Reproducibility:** Ease of cloning or generating the template and running the provided checks without guesswork.
- **Reflection and critical thinking:** Clear discussion of assumptions, risks, limitations, and possible improvements.

Task 7 - Geospatial RAG + Knowledge Graph Dashboard

Background

A single environmental question such as "how has urban heat stress changed in this region over the last decade?" may require combining satellite-derived raster data, scientific literature, administrative boundary vectors, and sensor time series.

The goal of this task is to design and partially implement a platform that connects geospatial data, a knowledge graph, and RAG and serves the result as an interactive narrative dashboard. Because the full vision is large, **scoping is a core part of the task**: we expect you to define and justify what you build versus what you describe to a maximum of about 8hrs.

Task

Design and build a scoped but working vertical slice of a geospatial RAG + Knowledge Graph platform. Your system should address all three layers, but the depth of implementation in each is your decision:

- **Geospatial data layer.** Ingest at least one real geospatial source.
- Optional Resources: <https://docs.overturemaps.org/examples/overture-tiles/> or <https://protomaps.com/api>
- **Knowledge graph layer.** Model the relationships between your data sources in a graph. At minimum, represent: datasets and their spatial extents, scientific reports or documents and the regions they describe. The graph does not need to be large.
- Optional resources: <https://github.com/oxigraph/oxigraph>
- **RAG layer.** Index at a few documents (scientific reports, dataset descriptions, or similar) and wire them to the graph so that a query can traverse graph edges to identify relevant documents before retrieving chunks. The final answer should be generated by an LLM of your choice and should cite specific sources.
- Optional resources: <https://github.com/pydantic/pydantic-ai>, <https://github.com/qdrant/qdrant>, and <https://openrouter.ai/?ref=openrouter.com>
- **Dashboard.** Build a minimal interactive dashboard that presents at least one narrative combining a map view, a graph-derived context panel, and an LLM-generated answer to

a user question. The map does not need to be a full GIS client, a rendered tile layer with an overlay is sufficient.

- Optional resources: <https://github.com/andfanilo/streamlit-leaflet>

Begin your submission with a **scope document** (half a page) stating: what you chose to implement fully, what you stubbed or mocked, what you deliberately left out, and why.

Format

Present your work in a repository (GitHub or GitLab). The project must be reproducible, well-documented, and include a clear README with setup instructions. Any data dependencies should either be included in the repo, fetched by a setup script, or clearly documented with download links. Include the scope document as SCOPE.md at the root.

What We Will Be Looking At

- **Scope document:** is the scoping decision explicit, justified, and consistent with what was actually built? Candidates who overpromise and underdeliver score lower than candidates who scope tightly and execute cleanly.
- **Graph model:** are the node and relation types meaningful? Does the graph reflect real relationships between geospatial data and documents, or is it a flat index dressed up as a graph?
- **RAG grounding:** does the graph actually influence retrieval, or is it bypassed? Answers should be traceable to specific documents and spatial context.
- **Geospatial handling:** is the spatial data queried and rendered correctly? We are not looking for a full GIS stack, but bounding boxes and projections should be handled without errors.
- **Dashboard coherence:** do the map, graph context, and generated narrative feel like parts of one tool, or three disconnected demos stitched together?
- **Reproducibility:** can we run the full system from a clean checkout on our own machine?